SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

CO1 – ASSESSMENT TOOL 1

APPLICATION BASED QUESTIONS

Course Code	DSA0311	**Course Title**	Natural Language Processing
CO Assessed	CO1 – Imitation (L1)	**Assessment**	Assessment Tool 1 – Application Based Questions
Weightage	40% of CIA		
CO1 Statement	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and interpret language processing. (BL-3)		
Student Name	A.ARCHISHMAN	**Reg. No.**	192425341
Section / Batch	2024	**Date**	27-10-2026

Code of Conduct

I, A.ARCHISHMAN (192425341), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ARCHISH

Instructions

- Develop a modular and efficient Python program that satisfies all functional requirements specified in each problem statement.
- Demonstrate correctness using suitable test cases and clearly show the output.
- Follow good programming practices, meaningful variable names, proper indentation, and comments where applicable.
- Duration: 30 minutes.

Assessment Sections

Section / Topic	Focus	Q. No.
Information Extraction	Regex-Based Resume Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

- Extract the candidate's name.
- Identify email addresses.
- Extract mobile numbers.
- Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
- Extract years of experience.
- Generate a structured summary of the candidate's profile.
- Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Aim

To implement a Regex-based resume parser that extracts candidate details and technical skills, identifies experience, and shortlists candidates with at least two years of experience and Python skill.

Python Program

```
import re

resumes = [
    """Name: Kavya Reddy
Email: kavya.reddy@example.com
Mobile: 9876543210
Skills: Python, SQL, Machine Learning
Experience: 3 years""",
    """Name: Rohit Kumar
Email: rohit.kumar@example.com
Mobile: 9123456780
Skills: Java, NLP
Experience: 4 years""",
    """Name: Meena Das
```

```
Email: meena.das@example.com
Mobile: 9988776655
Skills: Python, NLP
Experience: 1 year"""
]

required_skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"]

for resume in resumes:
    name = re.search(r"Name:\s*(.+)", resume).group(1)
    email = re.search(r"[\w.-]+@[\w.-]+\.\w+", resume).group()
    mobile = re.search(r"\b\d{10}\b", resume).group()
    experience = int(re.search(r"Experience:\s*(\d+)", resume).group(1))
    skills_text = re.search(r"Skills:\s*(.+)", resume).group(1)
    skills = [skill for skill in required_skills
              if re.search(re.escape(skill), skills_text, re.I)]
    eligible = experience >= 2 and any(s.lower() == "python".lower()
                                       for s in skills)

    print(f"Name: {name}")
    print(f"Email: {email}")
    print(f"Mobile: {mobile}")
    print(f"Skills: {', '.join(skills) if skills else 'None'}")
  print(f"Experience: {experience} year(s)")
    print(f"Eligible: {'Yes' if eligible else 'No'}")
    print("-" * 35)
```

Output

```
Name: Kavya Reddy
Email: kavya.reddy@example.com
Mobile: 9876543210
Skills: Python, SQL, Machine Learning
Experience: 3 year(s)
Eligible: Yes
-----------------------------------
Name: Rohit Kumar
Email: rohit.kumar@example.com
Mobile: 9123456780
```

```
Skills: Java, NLP
Experience: 4 year(s)
Eligible: No
-----------------------------------
Name: Meena Das
Email: meena.das@example.com
Mobile: 9988776655
Skills: Python, NLP
Experience: 1 year(s)
Eligible: No
-----------------------------------
```

Result

The program extracts names, email addresses, mobile numbers, technical skills, and years of experience from sample resumes. It correctly shortlists Kavya Reddy because she has Python skill and three years of experience. The other candidates do not satisfy both eligibility conditions.

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

- Search products using an exact keyword.
- Perform prefix-based searches.
- Perform suffix-based searches.
- Search using partial keywords.
- Perform case-insensitive searches.
- Display all matching product names.
- Generate a report showing the total number of matching products for each search.

Aim

To create a product search system using Regex that supports exact, prefix, suffix, partial, and case-insensitive matching, and reports matching product names and counts.

Python Program

```
import re

products = [
    "Apple iPhone 15",
    "Apple MacBook Air",
    "Samsung Galaxy S24",
    "Samsung Galaxy Buds",
    "Logitech Wireless Mouse",
    "Gaming Mouse Pad",
    "Dell Inspiron Laptop"
]

def search_products(keyword, search_type):
    patterns = {
        "exact": rf"^{re.escape(keyword)}$",
        "prefix": rf"^{re.escape(keyword)}",
        "suffix": rf"{re.escape(keyword)}$",
        "partial": re.escape(keyword)
    }
```

```
    pattern = re.compile(patterns[search_type], re.IGNORECASE)
    matches = [product for product in products if pattern.search(product)]
    print(f"Search: {keyword} | Type: {search_type}")
    print("Matches:", ", ".join(matches) if matches else "No matches found")
    print("Total matches:", len(matches))
    print()

search_products("Samsung Galaxy S24", "exact")
search_products("Apple", "prefix")
search_products("Mouse", "suffix")
search_products("Galaxy", "partial")
```

Output

```
Search: Samsung Galaxy S24 | Type: exact
Matches: Samsung Galaxy S24
Total matches: 1

Search: Apple | Type: prefix
Matches: Apple iPhone 15, Apple MacBook Air
Total matches: 2

Search: Mouse | Type: suffix
Matches: Logitech Wireless Mouse
Total matches: 1

Search: Galaxy | Type: partial
Matches: Samsung Galaxy S24, Samsung Galaxy Buds
Total matches: 2
```

Result

The program successfully performs exact, prefix, suffix, and partial product searches without case sensitivity. It displays all matching product names and the total count for each search.

Section 3: Regular Expressions – University Registration System

3. A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

- Validate the student register number.
- Validate the institutional email address.
- Validate the course code.
- Validate the semester information.
- Validate the student's mobile number.
- Display appropriate validation messages for each field.
- Generate a final registration status report indicating whether the registration is successful.

Aim

To validate a student's register number, institutional email, course code, semester, and mobile number using Regex, then display field-wise messages and the final registration status.

Python Program

```
import re

student = {
    "Register Number": "192425341",
    "Email": "archishman@simats.edu.in",
    "Course Code": "DSA0311",
    "Semester": "Semester 3",
    "Mobile": "9876543210"
}

patterns = {
    "Register Number": r"^\d{9}$",
    "Email": r"^[\w.-]+@simats\.edu\.in$",
    "Course Code": r"^[A-Z]{3}\d{4}$",
    "Semester": r"^Semester ([1-8])$",
    "Mobile": r"^\d{10}$"
```

```
}

all_valid = True
print("UNIVERSITY REGISTRATION VALIDATION")
for field, value in student.items():
    valid = re.fullmatch(patterns[field], value) is not None
    print(f"{field}: {value} - {'Valid' if valid else 'Invalid'}")
    all_valid = all_valid and valid

print("\nFINAL REGISTRATION STATUS")
print("SUCCESSFUL" if all_valid else "FAILED - Please correct invalid fields.")
```

Output

```
UNIVERSITY REGISTRATION VALIDATION
Register Number: 192425341 - Valid
Email: archishman@simats.edu.in - Valid
Course Code: DSA0311 - Valid
Semester: Semester 3 - Valid
Mobile: 9876543210 - Valid

FINAL REGISTRATION STATUS
SUCCESSFUL
```

Result

The program validates all five registration fields using regular expressions. Since the sample register number, institutional email, course code, semester, and mobile number match their patterns, the final registration status is SUCCESSFUL.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation
Application of Concepts	30	Accurately applies appropriate concepts	Applies concepts correctly with minor errors	Basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, efficient approach	Mostly logical with minor gaps	Basic approach with inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct with justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, organized, properly explained	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented

Assessment scoring table

Q. No.	Understanding	Application	Problem-Solving	Accuracy	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____